

Approximating Functions: A Practical Guide to Sollya and Function Implementation

CCMATH

1 The core problem

A handful of operations are *correctly rounded* on every conforming platform: addition, subtraction, multiplication, division, square root, and fused multiply-add are required by IEEE 754 to behave as if the exact result were computed to infinite precision and rounded once [8]. Almost everything else cannot be handled that way.

$\exp$, $\log$, $\sin$, $\cos$, atan , pow , tgamma , and the rest are transcendental or algebraic-but-irrational. There is no finite formula in terms of the basic operations that yields their value exactly.

Implementing such a function is an engineering process with four recurring stages [10, 11]:

- (1) **Argument reduction.** Use mathematical identities to map the input into a small, well-behaved interval.
- (2) **Approximation.** Replace the true function by a polynomial whose error is small and bounded by a proof.
- (3) **Reconstruction.** Combine the approximation with information discarded during reduction to recover the full-range result.
- (4) **Rounding.** Deliver the result correctly (or faithfully) rounded in the target format, in every rounding mode the function supports.

Stages 1, 3, and 4 are function-specific algebra and careful floating-point bookkeeping. Stage 2 is where Sollya is used.

2 Polynomial approximation

Polynomials are the natural tool. They require only the four basic operations, map well to hardware FMA, and their error can be bounded by analysis.

Choosing coefficients so the error is small over an interval is the approximation problem. The wrong choice of norm or reference polynomial can make the bound much worse than necessary.

3 Taylor approximation

A Taylor series centered at zero has the form

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!} x^k.$$

Truncating at degree n gives a polynomial whose error is smallest at $x = 0$ and grows toward the interval edges.

A Taylor polynomial can look excellent in a small hand test and still be wrong for the interval. The worst input on the interval controls the design.

For a fixed-degree polynomial in a fixed format, the usual choice is minimax approximation.

4 Minimax approximation

A minimax polynomial minimizes the largest error on an interval.

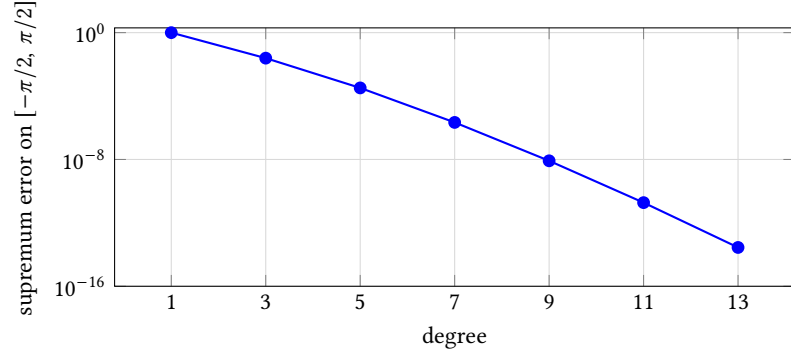


Fig. 1. Supremum absolute error of odd Taylor sine polynomials on $[-\pi/2, \pi/2]$ as a function of degree. Adding terms reduces the error, but the distribution is worst-case at the interval endpoints.

For many elementary functions, this is the right goal. A minimax polynomial spreads error across the interval instead of spending most of the accuracy near one point.

For a degree n polynomial, the best minimax approximation has an error curve that reaches its maximum size at least $n + 2$ times, with alternating signs [10].

The theorem gives a useful check. If the error plot has one large spike and many small regions, the polynomial is probably not close to minimax. If the peaks are balanced, the approximation is more likely to use the degree well.

Taylor approximation is local. Minimax approximation is worst-case driven.

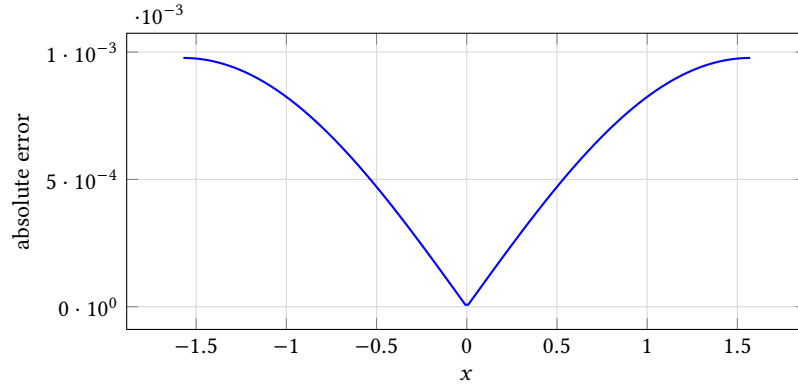


Fig. 2. Absolute error of the degree 5 Taylor approximation to $\sin(x)$ on $[-\pi/2, \pi/2]$. The error is smallest near zero and grows toward the interval edges.

5 Remez

The Remez exchange algorithm is the usual numerical method for computing a minimax polynomial.

Remez repeatedly finds error peaks, adjusts the polynomial so that those peaks are more evenly spaced, and then repeats until the error curve has the minimax shape [10].

Sollya provides a `remez` command for this.

There is one important limit. Remez gives a polynomial with real-number coefficients. Source code cannot store arbitrary real numbers. It stores float, double, long double, fixed-width constants, or pairs such as double-double values.

A real-coefficient Remez polynomial is useful for exploration. It is not automatically the polynomial that belongs in the source code.

6 Coefficients must fit the machine

If a real-coefficient minimax polynomial is rounded by hand, the result may no longer have the minimax error shape. The error can grow enough to break the intended bound [10].

Sollya's `fpmminimax` command addresses this problem.

`fpmminimax` searches for a near-minimax polynomial whose coefficients are already constrained to the requested formats [10, 11].

Use the commands this way:

Step	Use
<code>remez</code>	Explore the real-coefficient approximation
<code>fpmminimax</code>	Generate coefficients for source code

Do not hand-round production coefficients unless the rounded polynomial is checked again.

A bad constant can ruin a good algorithm. A decimal literal that looks harmless may round differently than the value Sollya produced. Prefer hexadecimal constants when the value is meant to be exact in the target format.

7 Absolute error and relative error

Choose the error model before generating a polynomial.

Absolute error is:

$$|p(x) - f(x)|$$

Relative error is:

$$\left| \frac{p(x)}{f(x)} - 1 \right|$$

Floating-point accuracy is often measured in ULPs. ULP size changes with the size of the result [11]. Because of that, relative error is often the right target.

Relative error is not always safe. If $f(x)$ is close to zero, dividing by $f(x)$ can make the relative error unstable. Change the kernel. Section 10 lists the usual transformed kernels.

8 Sollya

Sollya is the main approximation tool used in this workflow.

Use Sollya for these tasks:

8.1 Running Sollya

Run a script:

```
sollya script.sollya
```

Start scripts like this:

```
prec = 200!;
display = hexadecimal!;
```

Table 1. Sollya commands used in this guide.

Task	Sollya command
Estimate the needed degree	guessdegree
Compute a real-coefficient minimax polynomial	remez
Compute machine-format coefficients	fminimax
Quickly estimate worst-case error	dirtyinfnorm
Compute a safer error bound	infnorm or supnorm
Print constants for source code	display = hexadecimal
Print Horner form	horner
Print expanded form	canonical

The precision setting controls Sollya's working precision. It should be higher than the target format. For double coefficients, 200 bits is a reasonable starting point.

Hexadecimal output matters. It lets C++ store the same floating-point value that Sollya printed. Decimal constants can add another rounding step.

9 A small Sollya example

Assume $\exp(x)$ has already been reduced to $\exp(r)$, where r is close to zero.

A common shape is:

$$\exp(x) = 2^k \exp(r)$$

where r lies in an interval such as:

$$\left[-\frac{\log(2)}{2}, \frac{\log(2)}{2} \right]$$

A first Sollya script:

```
prec = 200!;
display = hexadecimal!;

f = exp(x);
I = [-1/2 * log(2); 1/2 * log(2)];

guessdegree(f, I, 1b-53);

p = fminimax(f, 6, [|D...|], I, relative);

eps_fast = dirtyinfnorm(p / f - 1, I);
print("approx_relative_error:", eps_fast, "bits:", log2(eps_fast));

supnorm(p, f, I, relative, 1b-40);
```

Read $\log_2(\text{error})$ as a rough number of correct bits. If the error is around 2^{-53} , the approximation is near double precision.

This only bounds the polynomial approximation for $\exp(r)$ on the reduced interval. It does not prove $\exp$.

The full function still needs reduction, floating-point evaluation, reconstruction, special cases, and final rounding.

10 Choosing the kernel

The kernel is the smaller function approximated after reduction.

It is not always the public function.

For $\sin(x)$, the kernel may be $\sin(r)$.

For small x , it may be better to approximate $\sin(r)$ divided by r .

For $\log 1p(x)$, it may be better to approximate $\log(1 + r)$ divided by r .

For $\expm1(x)$, it may be better to approximate $\expm1(r)$ divided by r .

The choice matters. A good kernel removes cancellation, avoids bad relative error near zero, and reduces the polynomial degree.

Use the algebra of the function before asking Sollya for coefficients.

What should be small after reduction, and what should be exact? That choice often determines the kernel.

10.1 Small-input transforms

$$\log(1 + x) = x \cdot \frac{\log(1 + x)}{x}$$

$$\exp(x) - 1 = x \cdot \frac{\exp(x) - 1}{x}$$

$$\sin(x) = x \cdot \frac{\sin(x)}{x}$$

$$\cos(x) = 1 - 2 \sin^2\left(\frac{x}{2}\right)$$

These forms are useful near zero. They factor out the part that causes cancellation or bad relative error. The implementation approximates the smoother quotient or uses a reconstruction identity that keeps more useful bits.

The identity must still be checked on the target interval and under the target rounding policy. None of these forms is automatically best on every interval.

11 Worked example: a small sine kernel

Start here for hands-on work

If you want to run Sollya and follow code in order, begin with Section 11.1 and Section 11.2. Skim Sections 1–7 first if the approximation vocabulary is new.

This example implements a restricted function:

$$\sin_small(x) \approx \sin(x) \quad \text{for } |x| \leq \frac{1}{16}$$

It is not a full implementation of $\sin$. It has no quadrant logic, no large-input reduction, and no special-case table. The whole kernel fits on one screen.

11.1 Set up Sollya

The worked examples need Sollya only. Install it once, then run the scripts under `sollya/` from this guide directory:

```
# macOS
brew install sollya

# Debian/Ubuntu
sudo apt install sollya

sollya sollya/sin_small_error.sollya
```

The build script runs the same scripts when you rebuild the guide PDF. MPFR, Gappa, Rocq, and SageMath are useful later for validation, but they are not required to follow the examples here.

11.2 Specify the target

Target:	$f(x) = \sin(x)$
Input interval:	$x \in \left[-\frac{1}{16}, \frac{1}{16}\right]$
Output type:	double
Accuracy target for this example:	well below 1 ULP on the stated interval under round-to-nearest

This example is only a teaching kernel. A full CCMath function would also need all standard special cases and all supported rounding modes.

11.3 Choose the kernel

Directly approximating $\sin(x)$ works on this interval, but the odd symmetry is useful. Since $\sin(x)$ is odd:

$$\sin(x) = xq(x^2)$$

where:

$$q(z) = \frac{\sin(\sqrt{z})}{\sqrt{z}}$$

Near zero, $q(z)$ is smooth and close to 1.

The implementation will compute:

$$z = x^2$$

and:

$$\sin(x) \approx x p(z)$$

where p approximates q .

11.4 Pick a simple polynomial

For a teaching example, the Taylor terms are easy to inspect:

$$\sin(x) = x - \frac{x^3}{6} + \frac{x^5}{120} - \frac{x^7}{5040} + \frac{x^9}{362880} + \dots$$

Factoring out x gives:

$$\sin(x) = x \left(1 - \frac{z}{6} + \frac{z^2}{120} - \frac{z^3}{5040} + \frac{z^4}{362880} + \dots \right)$$

where $z = x^2$.

For production work, use `fpmminimax` and record the bound. Here, the Taylor coefficients keep the example readable, and Sollya is used to check the error.

11.5 Check the approximation in Sollya

Use Sollya to bound the error over the actual interval:

```
prec = 200!;
display = hexadecimal!;

I = [-1/16; 1/16];

p = x * (
    0x1.0000000000000p+0
  + x^2 * (
    -0x1.5555555555555p-3
  + x^2 * (
    0x1.1111111111111p-7
  + x^2 * (
    -0x1.a01a01a01a01ap-13
  + x^2 * 0x1.71de3a556c734p-19
    )
  )
)
);

abs_err = supnorm(p, sin(x), I, absolute, 1b-80);
rel_err = supnorm(p, sin(x), I, relative, 1b-80);

print("absolute_error:", abs_err);
print("relative_error:", rel_err);
```

The code is not accepted because the series looks familiar. It is accepted only after the interval and error are checked.

11.6 Implement the kernel

A compact C++ implementation can use Horner form:

```

[[nodiscard]] constexpr double sin_small(double x) noexcept
{
    const double z = x * x;

    const double p =
        (((0x1.71de3a556c734p-19 * z
          - 0x1.a01a01a01a01ap-13) * z
         + 0x1.1111111111111p-7) * z
         - 0x1.5555555555555p-3) * z
         + 0x1.0000000000000p+0;

    return x * p;
}

```

There is no range check in this kernel. The caller is responsible for sending only values in the promised interval. In a real internal implementation, that precondition should be enforced by the surrounding reduction path or by tests that call the kernel only on the reduced interval.

11.7 Test the kernel

For this restricted kernel, useful tests include:

- (1) x equals 0
- (2) x equals the interval endpoints
- (3) several values near the endpoints
- (4) positive and negative inputs with matching signs
- (5) comparison against MPFR or another high-precision oracle

A simple round-to-nearest test should compare the returned bit pattern or ULP distance, not printed decimal text.

Example values:

$$0, \quad \frac{1}{1024}, \quad -\frac{1}{1024}, \quad \frac{1}{32}, \quad -\frac{1}{32}, \quad \frac{1}{16}, \quad -\frac{1}{16}$$

The endpoints matter because the approximation was proved only on that interval. If a later change expands the interval to $[-1/8, 1/8]$, the proof and tests must be redone.

11.8 What this example leaves out

This small kernel does not implement full $\sin$.

A full implementation still needs:

- (1) reduction for arbitrary finite inputs
- (2) quadrant reconstruction
- (3) handling of NaN and infinities
- (4) signed-zero behavior
- (5) directed rounding behavior
- (6) large-input Payne-Hanek reduction
- (7) validation against an oracle

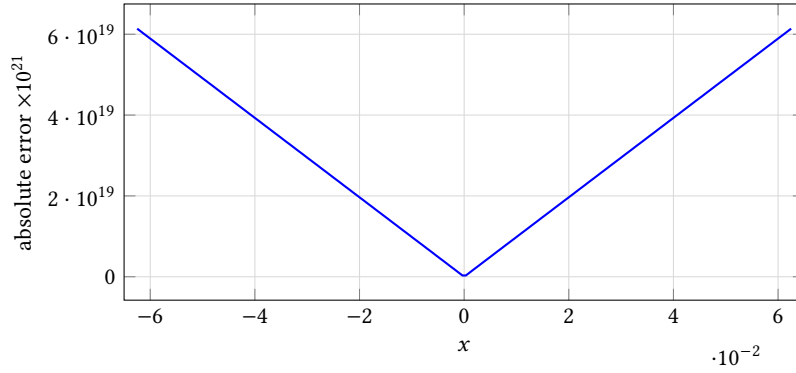


Fig. 3. Scaled absolute error of the small sine teaching polynomial on $[-1/16, 1/16]$. The plot shows the approximation error for the restricted kernel, not the error of a full $\sin$ implementation.

(8) a recorded error budget for reduction, evaluation, and reconstruction

The subsections above follow the same loop used inside larger functions. The larger versions add reduction, tables, multiple precision tiers, special cases, and harder rounding decisions.

12 Advanced worked example: a small $\log_2$ core

This example shows a more advanced shape than the small sine kernel. It uses a table reduction, a transformed kernel, split constants, and a polynomial on a small residual interval.

The goal is not to implement full $\log_2$. The example implements the core used after the input has already been decomposed as:

$$x = m2^e$$

with:

$$m \in [1, 2)$$

The full result is:

$$\log_2(x) = e + \log_2(m)$$

The core computes the $\log_2(m)$ part.

12.1 Choose the table reduction

Divide $[1, 2)$ into B buckets. For a teaching version, use:

$$B = 16$$

For each bucket, choose a value r_i close to the reciprocal of the bucket center. For an input mantissa m , compute:

$$u = mr_i - 1$$

If r_i is well chosen, u is small. The identity is:

$$m = \frac{1+u}{r_i}$$

so:

$$\log_2(m) = \log_2(1 + u) - \log_2(r_i)$$

The table stores r_i and a split form of $-\log_2(r_i)$.

One split is:

$$-\log_2(r_i) = h_i + \ell_i$$

The high part h_i can be rounded to fewer bits so that adding it to the exponent is exact in the fast path. The low part ℓ_i carries the remaining bits.

12.2 What the table buys

The table makes the polynomial interval smaller. With $B = 16$, a simple bucket-center choice gives a residual roughly bounded by:

$$|u| \leq \frac{1}{2B}$$

The exact bound depends on how the buckets and reciprocals are chosen. The point is that more table bits shrink the interval that the polynomial has to cover.

This is the usual tradeoff. A larger table costs memory and cache pressure. A smaller residual can reduce polynomial degree or make the error proof easier.

12.3 Approximate the residual function

After the table reduction, the remaining function is:

$$\log_2(1 + u)$$

Since u is small, approximate:

$$g(u) = \frac{\log_2(1 + u)}{u}$$

Then:

$$\log_2(1 + u) \approx u p(u)$$

This avoids a zero at $u = 0$. The function $g(u)$ is smooth near zero, with value $1/\log(2)$.

A Sollya script for the polynomial looks like this:

```
prec = 200!;
display = hexadecimal!;

U = [-1/32; 1/32];
g = log2(1 + x) / x;

p = fpminimax(g, 5, [|D...|], U, relative);

abs_err = supnorm(x * p, log2(1 + x), U, absolute, 1b-80);
rel_err = supnorm(p, g, U, relative, 1b-80);

print("absolute_residual_error:", abs_err);
print("relative_kernel_error:", rel_err);
```

```
print("polynomial:");
horner(p);
```

This checks the part controlled by the polynomial. It does not check table indexing, the split constants, or the final addition with the exponent.

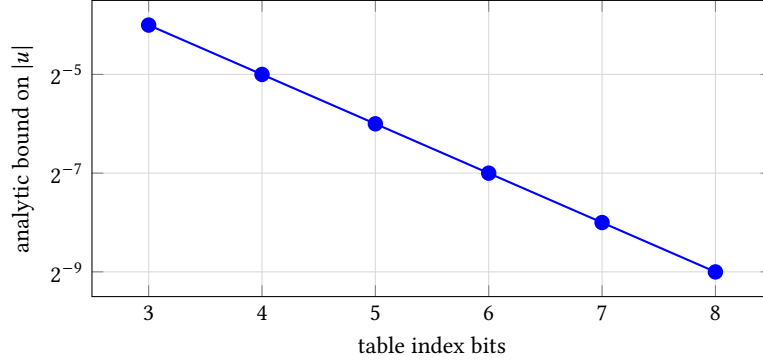


Fig. 4. Analytic residual bound after one reciprocal table reduction. More table bits shrink the interval used by the polynomial.

12.4 Generate the table

The table should be generated from the invariant, not typed by hand.

For each bucket, generate:

$$c_i = 1 + \frac{i + 1/2}{B}$$

$$r_i = \text{round}_D\left(\frac{1}{c_i}\right)$$

$$a_i = -\log_2(r_i)$$

Then split a_i into a high part and a low part:

$$h_i = \text{round}_t(a_i)$$

$$\ell_i = \text{round}_D(a_i - h_i)$$

Here t is the chosen precision for the high part. It should be chosen so that the later addition with the exponent is exact for the path being proved.

A Sollya table script can be shaped like this:

```
prec = 200!;
display = hexadecimal!;

B = 16!;
hi_bits = 32!;

for i from 0 to B - 1 do {
```

```

c = 1 + (i + 0.5) / B;
r = round(1 / c, D, RN);
a = -log2(r);
hi = round(a, hi_bits, RN);
lo = round(a - hi, D, RN);
print("{", r, ",", hi, ",", lo, "},");
};

```

The generated log should record B , hi_bits , the residual interval, and the Sollya version.

12.5 Kernel shape

The implementation should make the data flow visible. The following code shows the shape without embedding generated table values in the guide:

```

struct Log2TableEntry
{
    double r;
    double hi;
    double lo;
};

struct Log2Poly
{
    double c1;
    double c2;
    double c3;
    double c4;
    double c5;
};

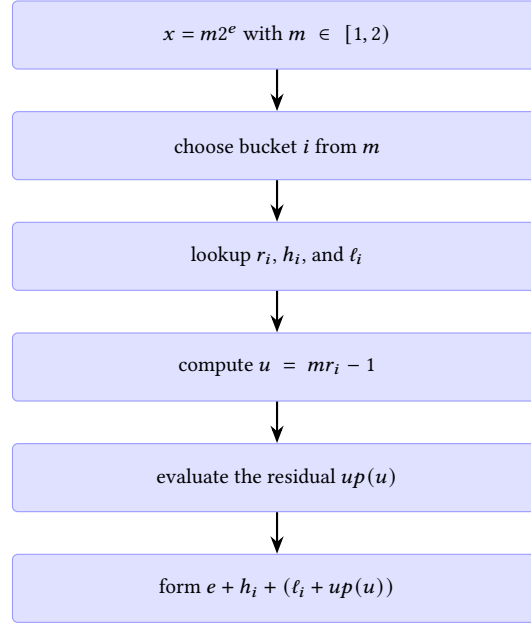
[[nodiscard]] constexpr double log2_core_small(double m,
                                                int e,
                                                const Log2TableEntry* table,
                                                const Log2Poly coeff) noexcept
{
    const int i = static_cast<int>((m - 1.0) * 16.0);
    const Log2TableEntry entry = table[i];

    const double u = std::fma(m, entry.r, -1.0);

    const double p =
        (((coeff.c5 * u + coeff.c4) * u + coeff.c3) * u + coeff.c2) * u
        + coeff.c1;

    const double residual = u * p;

```

Fig. 5. Data flow in the small $\log_2$ core.

```

const double tail = entry.lo + residual;

return (static_cast<double>(e) + entry.hi) + tail;
}

```

The coefficient object is produced from the Sollya polynomial. The table object is produced by the table-generation script. Production code should store those generated values directly in the implementation or in the nearby generated constants file.

12.6 What must be proved

This example has more moving parts than the small sine kernel. The proof needs bounds for:

- (1) table index selection
- (2) reciprocal-table rounding
- (3) the size of u
- (4) the polynomial approximation to $\log_2(1 + u)$
- (5) the floating-point evaluation of $up(u)$
- (6) the split addition $e + h_i + \ell_i$
- (7) the final rounding under each supported mode

A Gappa proof is a good fit for the straight-line part after the table entry has been chosen. MPFR testing should cover bucket boundaries, values where u is near its largest magnitude, exact powers of two, subnormals routed through the full log path, and every supported rounding mode.

12.7 What this example leaves out

This is not a complete $\log_2$ implementation.

A full function still needs:

- (1) bit decomposition for all finite positive inputs
- (2) handling of zero, negative inputs, NaN, and infinities
- (3) subnormal normalization
- (4) a larger or more carefully designed table
- (5) directed rounding checks
- (6) an accurate path for hard cases if correct rounding is the target

The example shows the shape used by real kernels: reduce with a table, keep the residual small, approximate a smooth transformed function, split constants for exactness, and prove the operation sequence that is actually compiled.

13 The sign, exponent, and significand split

A normal finite binary floating-point number can be written as [8, 11]:

$$x = s_x \cdot 2^{e_x} \cdot (1 + m_x)$$

This decomposition applies to finite nonzero normal binary floating-point numbers.

- s_x is the sign, either +1 or -1
- e_x is the unbiased binary exponent
- m_x is the fractional part of the significand
- $1 + m_x$ is the normalized significand and lies in $[1, 2)$

For example:

$$6.5 = 1.625 \cdot 2^2$$

so:

$$s_x = +1$$

$$e_x = 2$$

$$m_x = 0.625$$

$$6.5 = +1 \cdot 2^2 \cdot (1 + 0.625)$$

In binary:

$$6.5 = 110.1_2 = 1.101_2 \cdot 2^2$$

The exponent carries scale. The significand stays in a fixed interval. This turns a problem over many magnitudes into a smaller approximation problem plus exponent bookkeeping.

The $(1 + m_x)$ form is for normal numbers. Subnormal numbers do not have the implicit leading 1 and need normalization or special handling. Zero, infinity, and NaN do not fit this decomposition [8]. Negative bases for pow need integer-exponent handling before any logarithmic path.

14 Argument reduction

Approximation works well only on a small interval. Argument reduction is how the implementation gets there [10, 11].

For exp:

$$\exp(x) = 2^k \exp(r)$$

where:

$$k \approx \text{round}\left(\frac{x}{\log(2)}\right)$$

and:

$$r = x - k \log(2)$$

The integer k carries the power-of-two scale. The residual r is small enough for a short polynomial.

For $\exp_2$, the split is simpler:

$$2^x = 2^k 2^r$$

where:

$$k = \lfloor x \rfloor$$

and:

$$r = x - k$$

The reconstruction step puts 2^k back into the exponent field when the result is in range.

For $\log_2$, the sign, exponent, and significand split gives:

$$\log_2(x) = e_x + \log_2(1 + m_x)$$

For natural log:

$$\log(x) = e_x \log(2) + \log(1 + m_x)$$

The exponent term is exact integer bookkeeping. The hard part is approximating the significand term, where $(1 + m_x)$ lies in a fixed interval for normal inputs.

This is why log implementations usually extract the exponent first, reduce the significand, and approximate only a small residual.

For trig functions, the usual reduction is modulo $\pi/2$ or $\pi/4$, followed by quadrant reconstruction.

Trig reduction is hard for large inputs. π is irrational. If x is huge and close to a multiple of $\pi/2$, subtracting $k\pi/2$ can lose the useful bits.

14.1 Cody-Waite reduction

Cody-Waite reduction stores the reduction constant as a sum of machine numbers:

$$C = C_{\text{hi}} + C_{\text{mid}} + C_{\text{lo}}$$

$$r = x - kC_{\text{hi}} - kC_{\text{mid}} - kC_{\text{lo}}$$

The split lets the subtraction keep more useful bits than one rounded constant would keep. The exact split depends on the function, interval, and target precision.

14.2 Payne-Hanek reduction

Payne-Hanek reduction uses many precomputed bits of the reduction constant and a higher-precision multiplication [10].

This is the large-input path for trig functions. Ordinary double arithmetic is not enough to reduce every double modulo $\pi/2$.

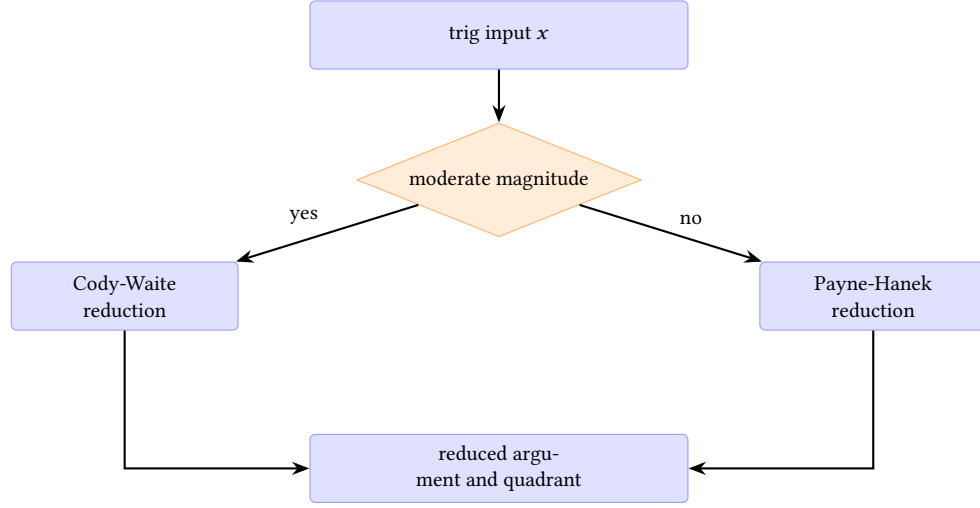


Fig. 6. Typical trig reduction split between moderate and large inputs.

14.3 Trig quadrant reconstruction

$$x = k \frac{\pi}{2} + r$$

The reducer chooses an integer k and a small residual r . The value of $k \bmod 4$ decides which small kernel reconstructs the result [10].

$k \bmod 4$	$\sin(x)$ reconstruction
0	$\sin(r)$
1	$\cos(r)$
2	$-\sin(r)$
3	$-\cos(r)$

This is why a full trig function is not just one polynomial. The reducer and quadrant logic decide which kernel is used and which sign is returned.

14.4 Reduction error

Reduction has its own error.

If the polynomial was proved for an exact reduced argument r , but the implementation computes r with too much error, the proof no longer applies.

Decide the total error target first. Give part of that budget to reduction. The reducer should carry enough extra precision that its error stays below that budget.

For many functions, this means split constants or a double-word reduced argument. For very large trig inputs, it can mean a separate large-argument reducer.

15 Evaluating the polynomial

After Sollya gives coefficients, the implementation still has to evaluate the polynomial in floating point.

The mathematical polynomial and the C++ expression are not identical. Every floating-point operation can round.

15.1 Horner evaluation

Horner evaluation is the usual default:

$$p(x) = c_0 + x(c_1 + x(c_2 + x(\dots)))$$

It uses few operations and maps well to FMA [10, 11].

With FMA, one step can be:

```
r = fma(r, x, c);
```

That rounds the multiply-add once.

15.2 Estrin evaluation

Estrin evaluation groups terms so more work can happen in parallel [11].

For example:

$$c_0 + c_1x + c_2x^2 + c_3x^3$$

can be grouped as:

$$(c_0 + c_1x) + x^2(c_2 + c_3x)$$

This can be faster on wide cores. It can also need more temporaries. Use it when measurements show that it helps.

15.3 Double-word evaluation

Some kernels need more precision in part of the evaluation.

Common tools include Fast2Sum, 2Sum, 2MultFMA, Veltkamp-Dekker splitting, double-double arithmetic, and compensated Horner evaluation [10, 11].

Do not use double-double everywhere by default. Use it where the error budget needs it.

The most useful extra precision is usually local. It protects one reduction, one product, or one reconstruction step. Treat it as a tool for a specific bound, not as a blanket fix.

16 Fast precision and accurate precision

Many elementary functions use more than one precision tier.

The fast path should be cheap. It handles ordinary inputs where a modest amount of extra precision is enough. The accurate path handles cases where the result is too close to a rounding boundary, or where the fast path cannot prove the final rounding decision.

For float functions, the implementation can often use double or double-double arithmetic for the fast path. Some cases may need a wider accurate path.

For double functions, the fast path usually needs a double-double-like computation. That does not mean every operation should behave like a complete higher-precision type. A full double-double type that renormalizes after every operation is often too expensive for the hot path.

A faster pattern is to arrange the computation so that important intermediate pieces are exact, or have a known error, and normalize only where the bound needs it. This is less general than a full double-double type, but it fits a fixed numerical kernel better.

The accurate path may use a dyadic floating-point type with a fixed integer significand, such as a 128-bit or 256-bit dyadic format. This gives a large exponent range and controlled precision without relying on hardware quad precision.

Quad precision and double-double are not interchangeable.

Quad precision has a much larger exponent range than double-double. A pair of doubles gives extra significand bits, but it does not cheaply give the exponent range of a true quad format. A dyadic format can be a better accurate-path type when the algorithm needs wide range and predictable fixed precision.

One tiering pattern is:

Tier	Role
double or double-double-like	Fast path
128-bit dyadic	First accurate path
256-bit dyadic	Hard-case path when needed

This is especially relevant for pow. A double pow implementation is harder than a float pow implementation because the final result has a 53-bit significand. The intermediate log, multiply, and exp reconstruction must leave enough margin for correct rounding.

17 Lookup tables

Lookup tables in elementary functions are not only cached constants.

A table entry usually exists to make one reduction step cheaper, more accurate, or both. For log and pow, common table values include approximations to quantities like $-\log_2(r)$, split into pieces.

A split table might store:

$$a = -\log_2(r)$$

and:

$$a = a_{\text{hi}} + a_{\text{mid}} + a_{\text{lo}}$$

The split is chosen for a reason. A high part may be rounded to a smaller precision so that a later multiplication is exact, or so that adding it to an exponent does not lose important bits.

Table generation should start from an invariant, not from a literal list of constants.

A table-generation note should record:

- (1) how the index is chosen
- (2) what real value each entry approximates
- (3) how the entry is split
- (4) which later operation is meant to be exact or tightly bounded
- (5) what interval remains after the table reduction
- (6) which precision tier uses the table

For powf, the fast path may use tables split into double-double or triple-double pieces because the target result is float. For pow in double, the accurate paths may need tables generated directly for the wider dyadic type used in that path.

Do not assume one table layout fits every precision tier.

If the accurate pass uses a 128-bit dyadic format, generate a table suited to that format. If there is a 256-bit hard-case path, generate the table for that path as well. The values may represent the same mathematical constants, but the split, precision, and exactness constraints can be different.

A table without its generation rule is hard to audit. A table with its invariant is part of the proof.

17.1 Log table reduction

$$\log_2(1 + m_x) = -\log_2(r_i) + \log_2(r_i(1 + m_x))$$

Define:

$$u = r_i(1 + m_x) - 1$$

Then:

$$\log_2(1 + m_x) = -\log_2(r_i) + \log_2(1 + u)$$

The table value r_i is chosen so that u is small. The polynomial only approximates $\log_2(1 + u)$ on a smaller interval. The table entry supplies the correction term $-\log_2(r_i)$.

This is the value of the table. It is not only avoiding a computation. It makes the residual small enough for a cheaper and more accurate polynomial. Section 12 uses the same pattern with bucket reciprocals and split $-\log_2(r_i)$ constants.

18 Why pow is harder than exp or log

pow combines the difficulties of log and exp.

For positive finite x , the usual shape is:

$$\text{pow}(x, y) = \exp(y \log(x))$$

$$\text{pow}(x, y) = 2^{y \log_2(x)}$$

$$y \log_2(x) = y(e_x + \log_2(1 + m_x))$$

This is why pow implementations often contain a log core, table reduction for the significand, split constants, and extra precision around the product $y \log_2(x)$. A small error in the log value can be magnified by y , then carried into the exp reconstruction.

That hides several problems.

First, $\log(x)$ needs its own range reduction, tables, approximation, and reconstruction.

Second, $y \log(x)$ can magnify error. If y is large, a small log error can become a large exponent error.

Third, exp has to reconstruct the final result, including overflow and underflow behavior.

Fourth, pow has many special cases. Negative bases, integer exponents, zeros, infinities, NaNs, signed zeros, odd and even integer exponents, and directed rounding modes all matter.

Because of this, pow is not a good first elementary function.

A better path is:

- (1) implement one exp-family function
- (2) implement one log-family function
- (3) learn the reduction, table, polynomial, and rounding structure

(4) then implement pow

pow is best understood as a composition of those two families, plus a large special-case layer and a harder final-rounding problem.

19 Compiler behavior

An error proof assumes a specific sequence of floating-point operations. The compiler may change that sequence unless the source and build settings prevent it.

The main hazards are:

Hazard	Why it matters
FMA contraction	$a*b + c$ may become one fused operation
Reassociation	fast-math may reorder arithmetic
Default rounding assumptions	directed modes may be ignored
Hidden precision	intermediates may have different width
Floating-point environment access	fenv support differs by compiler

These two expressions do not have the same rounding behavior:

```
a * b + c
```

```
std::fma(a, b, c)
```

The first can round the product and then the sum. The second rounds once.

If a proof needs FMA, use FMA explicitly. If a proof needs the live rounding mode, compile in a mode that respects it. Do not prove one operation sequence and compile a different one [11].

This also affects double-double arithmetic. Some exact multiplication algorithms for double-double arithmetic without FMA are only valid under round-to-nearest. Making them correct under directed rounding can require extra handling. That cost belongs in the design.

20 The error budget

$$E_{\text{total}} \leq E_{\text{reduce}} + E_{\text{approx}} + E_{\text{eval}} + E_{\text{reconstruct}} + E_{\text{round}}$$

- E_{reduce} comes from argument reduction
- E_{approx} comes from replacing the function with a polynomial or rational function
- E_{eval} comes from evaluating that approximation in floating point
- $E_{\text{reconstruct}}$ comes from scaling, sign changes, table corrections, and final algebra
- E_{round} is the margin needed for the final rounding decision

This inequality is a model for thinking about the implementation. It is not always the final tight proof. It is useful because it prevents a common mistake: proving the polynomial and forgetting the rest of the function.

21 Correct rounding

Correct rounding means the function returns the value required by the active rounding mode.

The exact result can land extremely close to a rounding boundary. In that case, the implementation needs enough extra precision to know which side of the boundary the exact result is on. This is called the Table Maker’s Dilemma [10, 11].

21.1 Faithful rounding

Faithful rounding is weaker than correct rounding.

It means the returned value is one of the two floating-point values surrounding the exact result.

Faithful rounding can be a useful interim target. It is not the same as correct rounding.

21.2 Ziv’s strategy

Ziv’s strategy uses a fast path and a fallback [10, 11]. Most inputs take the fast path. Rare hard cases take the accurate fallback. Correctly rounded libraries such as CRLibm use this pattern [4].

21.3 Hard cases

For some functions and formats, the hardest inputs are known from search. Algorithms such as the L algorithm and SLZ algorithm are used to find them [10, 11].

Knowing the worst case tells you how much precision is enough.

22 Special cases

Special cases are part of the function.

Handle NaNs, infinities, signed zeros, subnormals, exact results, overflow, underflow, invalid operations, and divide-by-zero behavior before the approximation path.

The polynomial kernel should only see inputs where its algebra is valid.

For each function, write the special-case table before writing the polynomial.

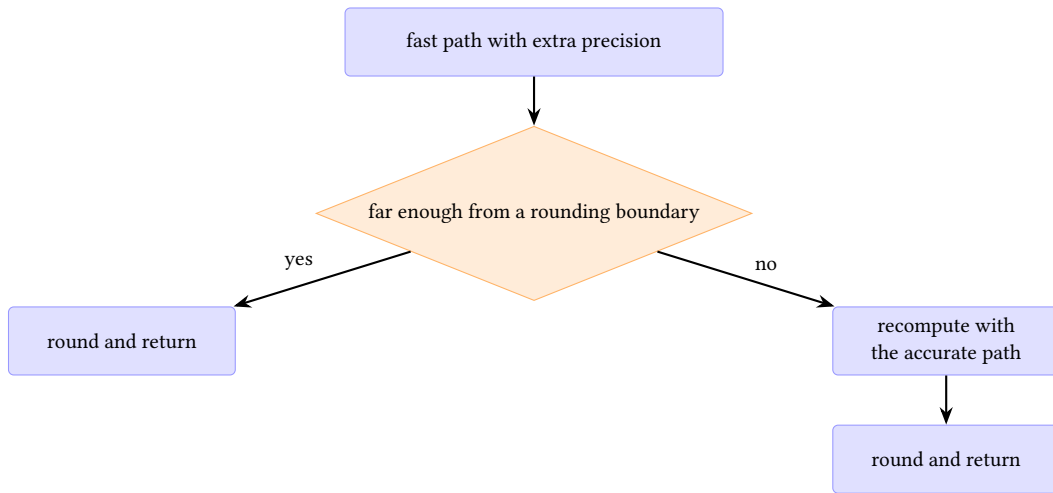


Fig. 7. Ziv’s strategy as a fast-path test with a precise fallback.

Special cases are often where an implementation first reveals whether it was designed from the contract or from the ordinary finite path. The finite path is only one part of the function.

23 CCMath rounding and standards policy

CCMath functions should match the C++ math contract for the function being implemented. That includes finite values, special cases, overload behavior, return type, NaNs, infinities, signed zeros, subnormals, overflow, underflow, and rounding modes.

For floating-point math functions, the target is correct behavior under these four IEEE rounding modes.

Mode	C fenv constant
Round to nearest, ties to even	FE_TONEAREST
Round toward positive infinity	FE_UPWARD
Round toward negative infinity	FE_DOWNWARD
Round toward zero	FE_TOWARDZERO

This affects dispatch.

A compiler builtin or platform libm call should not be assumed correct outside round-to-nearest. If a runtime path uses a builtin, SIMD implementation, or platform function that is only known safe under FE_TONEAREST, the dispatcher must check the live rounding mode first.

Directed modes should route to the generic kernel unless that fast path has been proved and tested for the active mode.

The public header should not bypass that guard at runtime. Constant evaluation may use an eligible `constexpr` builtin when it satisfies the same edge-case and accuracy policy. Runtime calls should go through the runtime dispatcher so the live rounding mode is observed.

This is part of the function contract. A polynomial that works under round-to-nearest is not automatically correct under FE_UPWARD, FE_DOWNWARD, or FE_TOWARDZERO. The final rounding step, signed-zero cases, overflow thresholds, and underflow behavior may depend on the active mode.

24 Tool roles

24.1 Sollya

Use Sollya to bound approximation error for a stated function, interval, error model, and coefficient format. That step does not cover reduction, evaluation, reconstruction, or final rounding.

24.2 Gappa

Use Gappa for floating-point error bounds in straight-line code.

This fits Horner kernels, Estrin kernels, compensated sums, split constants, FMA sequences, and reconstruction fragments where the operation order is fixed.

Sollya bounds the mathematical approximation error. Gappa helps bound the floating-point evaluation error.

24.3 Rocq

Rocq is an interactive theorem prover.

Use it when the proof needs a larger machine-checked structure: definitions, lemmas, interval facts, rounding facts, and the link between local Gappa proofs and the function-level theorem.

For CCMath, Rocq belongs to the high-assurance path, not the first pass for every function.

24.4 MPFR

Use MPFR as the independent oracle.

MPFR testing is not a proof, but it is the usual way to validate an implementation at scale. Test random inputs, boundary inputs, subnormals, overflow and underflow thresholds, exact cases, hard reduction cases, hard rounding cases, and every supported rounding mode.

Measure ULP error. Do not rely on printed decimal comparisons.

24.5 SageMath

Use SageMath for symbolic manipulation, series expansion, exact rational arithmetic, continued fractions, interval experiments, and prototype derivations.

For example, before writing a $\log 1p$ kernel, inspect:

$$\frac{\log(1+x)}{x}$$

Before writing a trig reducer, inspect continued-fraction approximants of $\pi/2$.

SageMath is a scratch tool. Sollya is the production coefficient path.

25 Checklist for a new function

25.1 Specify the target

Write down the function, supported formats, supported rounding modes, domain, special cases, accuracy target, and whether the goal is correct rounding, faithful rounding, or a stated ULP bound.

Do this before writing the approximation.

25.2 Choose the reduction

Decide how the input maps into a small interval.

Function	Typical reduction
$\exp(x)$	2^k times $\exp(r)$
$\log(x)$	mantissa and exponent split
trig	reduction modulo $\pi/2$ or $\pi/4$
$\text{pow}(x, y)$	special integer paths, logarithmic paths, exact cases

Also decide how much precision the reduction carries.

Record the sign, exponent, and significand split when the function uses it.

25.3 Define the kernel

Write the exact function that Sollya will approximate.

This may be $\sin(r)$, $\sin(r)/r$, $\exp(r)$, $\expm1(r)/r$, $\log(1+r)/r$, a table residual, or another transformed function.

25.4 Choose the interval

Write the interval explicitly.

$$r \in \left[-\frac{\log 2}{2}, \frac{\log 2}{2} \right]$$

$$r \in \left[-\frac{\pi}{4}, \frac{\pi}{4} \right]$$

The approximation proof only holds on the interval that was checked.

25.5 Choose the error model

Choose absolute error, relative error, or a weighted error.

Use relative error when it matches ULP behavior. Use absolute error or a transformed kernel when relative error is ill-conditioned.

25.6 Generate coefficients

Use `guessdegree` to estimate the degree. Then test nearby degrees.

Use `fpmimax` for production coefficients.

```
prec = 200!;  
display = hexadecimal!;
```

Generate coefficients in the same formats used by the implementation.

25.7 Generate the tables

For each lookup table, record the invariant.

At minimum, write down the index formula, the represented real value, the split format, the exactness constraint, the residual interval, and the precision tier that consumes the table.

Generate separate tables for separate precision tiers when the split or exactness constraint changes.

For log-style tables, record the residual formula, such as $u = r_i(1 + m_x) - 1$, and the correction term supplied by the table.

25.8 Check the approximation

Use `dirtyinfnorm` for quick iteration. Use `supnorm` for the recorded bound.

Record the interval, polynomial degree, coefficient formats, error model, and bound.

25.9 Choose the evaluation scheme

Choose Horner, Estrin, compensated Horner, double-word evaluation, or a hybrid.

This choice is part of the proof.

25.10 Choose the precision tiers

Decide which inputs use the fast path and which inputs need an accurate path.

For float, double or double-double may be enough for many fast paths.

For double, expect a double-double-like fast path and a wider accurate path for hard cases.

Use dyadic formats when the accurate path needs fixed precision and wider exponent range.

25.11 Prove the evaluation error

Bound the floating-point error from the actual operation sequence.

Use Gappa where possible. The approximation error alone is not enough.

Keep the total error budget visible. The polynomial bound is only one term.

25.12 Reconstruct and round

Undo the reduction. Handle scaling, quadrant selection, sign reconstruction, table lookup, compensated addition, overflow, underflow, and double-rounding hazards where they apply.

If the target is correct rounding, handle hard-to-round inputs or provide a bounded strategy that covers them.

25.13 Validate

Compare against MPFR over random, structured, boundary, subnormal, overflow, underflow, cancellation-heavy, and adversarial inputs.

Run every supported rounding mode.

25.14 Record the status

Document supported formats, supported rounding modes, maximum observed ULP, proven bound if one exists, unsupported cases, known gaps, test coverage, oracle used, and the relevant Sollya, Gappa, SageMath, or Rocq artifacts.

26 CCMath placement

A typical function needs special-case handling, reduction constants, approximation coefficients, an evaluation kernel, reconstruction logic, rounding-mode handling, oracle tests, and status notes.

Generated constants are part of the algorithm. If the interval, coefficient format, polynomial degree, table split, or evaluation order changes, the proof needs review.

Keep public APIs clean. Put function-specific machinery in internal support code. Keep kernels small enough to audit. Store or document enough information to regenerate the constants.

If a function is only faithful, say so. If only round-to-nearest has been validated, say so. If directed modes are not covered, say so.

Do not imply a stronger contract than the proof and tests support.

27 Other approaches

27.1 Traditional libm workflow

The usual workflow matches Figure 7, with explicit bounds on floating-point evaluation error before the final rounding argument. Sollya and Gappa fit this structure naturally.

27.2 RLIBM-style result approximation

RLIBM uses a different target [13].

The traditional route approximates the real function, then proves that rounding gives the desired floating-point result.

RLIBM generates polynomials that target the correctly rounded floating-point result directly. The polynomial is trying to land inside the interval of real values that round to the right floating-point answer.

Not the default CCMath workflow. Worth studying for correctly rounded float functions, generated libraries, and multi-format work.

27.3 CRLIBM and CORE-MATH

CRLIBM is useful for studying Ziv-test based correctly rounded functions [3, 4].

CORE-MATH is useful for modern correctly rounded elementary functions with compact implementations and proof-oriented notes [3].

Study the structure: reduction, approximation, reconstruction, hard cases, proof.

28 Minimal Sollya template

Start from the script in Section 9. For a reproducible production script, also print the certified bound and the coefficient forms:

```
eps_cert = supnorm(p, f, I, relative, 1b-40);
print("certified_relative_error:", eps_cert);

print("certified_relative_error_bits:", log2(eps_cert));

print("canonical:");
canonical(p);

print("horner:");
horner(p);
```

Keep the script with the implementation or proof tooling. Constants in source should be reproducible from the script.

29 Minimal SageMath scratch template

Use SageMath before the production Sollya script when the algebra is still being explored.

```
var('x')

f = log(1 + x) / x
series(f, x, 0, 10)
```

For continued fractions:

```
continued_fraction(pi / 2).convergents()[10]
```

For exact rational checks:

```
R = RationalField()
a = R(355) / R(113)
```

```
a - pi.n(100)
```

Treat these as exploration unless the exact claim is recorded and checked as part of the proof.

30 Citation policy

Citations use ACM numeric style with `ACM-Reference-Format.bst`. Prefer full author names and stable DOI or URL fields in the bibliography.

Source	Typical use in this guide
IEEE 754-2019 [8]	Required operations, rounding modes, exceptions, NaNs, infinities, signed zeros, subnormals
Muller, <i>Elementary Functions</i> [10]	Minimax approximation, Remez, range reduction, Cody-Waite, Payne-Hanek, Ziv tests
<i>Handbook of Floating-Point Arithmetic</i> [11]	ULPs, FMA, double-word arithmetic, compiler hazards, Gappa, MPFR
Sollya [16]	Command syntax and coefficient generation
Gappa [6]	Straight-line floating-point error proofs
Rocq [18]	Machine-checked proof structure
SageMath [14]	Symbolic exploration before production scripts
MPFR [17]	Oracle reference behavior
RLIBM [13]	Direct approximation of the correctly rounded result
Goldberg [7], Oracle NCG [12]	Introductory floating-point background

31 Further reading

Free resources come first. Paid books follow for deeper derivations and citable design notes. Each entry lists a URL and what it covers.

31.1 Free resources

31.1.1 First pass on floating point. Oracle reprint of Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

Introductory floating-point article. Covers rounding, guard digits, cancellation, ULPs, and IEEE arithmetic.

The Floating-Point Guide. <https://floating-point-gui.de/>

Short beginner reference on binary floating-point behavior, not libm design.

IEEE 754-2019 official page. <https://standards.ieee.org/standard/754-2019.html>

Official standard page. The full text is not free on every access path.

31.1.2 Sollya and approximation work. *Sollya official site*. <https://www.sollya.org/>

Tool site and current manual. Minimax approximation, constrained coefficients, and certified supremum-norm checks.

Sollya fpmimax documentation. <https://www.sollya.org/sollya-current/help.php?name=fpmimax>

Command reference for production coefficients in requested floating-point formats.

Sollya supnorm documentation. <https://www.sollya.org/sollya-current/help.php?name=supnorm>

Recorded approximation bounds. Run after quick dirtyinfnorm checks.

31.1.3 *Proof and validation tools. Gappa user guide.* <https://gappa.gitlabpages.inria.fr/gappa/index.html>

Straight-line floating-point and fixed-point error proofs after Sollya.

Rocq Prover reference manual. <https://rocq-prover.org/refman>

Machine-checked proof structure beyond local Gappa bounds.

GNU MPFR official site. <https://www.mpfr.org/>

High-precision oracle with correct rounding.

SageMath documentation. <https://doc.sagemath.org/>

Symbolic exploration before production Sollya scripts: series, exact rationals, and continued fractions.

31.1.4 *Implementations and current research. LLVM libc math status.* <https://libc.llvm.org/math/>

LLVM libc math accuracy table, including functions marked correctly rounded in all four rounding modes.

CORE-MATH project. <https://core-math.gitlabpages.inria.fr/>

Compact correctly rounded implementations and proof-oriented notes.

CORE-MATH paper. <https://inria.hal.science/hal-03721525v3/file/core-math-final.pdf>

Project goals and design direction.

RLIBM project page. <https://people.cs.rutgers.edu/~sn349/rllibm/>

Polynomials that target the correctly rounded floating-point result directly.

RLIBM-ALL paper. <https://arxiv.org/abs/2108.06756>

Multi-format, multi-rounding-mode RLIBM.

RLIBM-MultiRound paper. <https://arxiv.org/abs/2504.07409>

Rounding-mode switching cost and methods that avoid depending on the live mode.

CRlibm report. <https://ens-lyon.hal.science/ensl-01529804v1/document>

Correctly rounded elementary functions and Ziv-style design.

31.1.5 *Performance-oriented implementation references. SLEEF paper.* <https://www.shibatch.net/sleef/>

Portable vectorized elementary functions with explicit vector paths and accuracy targets [15].

LLVM libc math slides. <https://libc.llvm.org/math/>

Modern CPU instructions, FMA, and rounding instructions in LLVM libc math work.

Arm Optimized Routines. <https://www.arm.com/why-arm/technologies/compute-library>

Target-specific scalar and vector math routines for Arm CPUs [2].

Agner Fog optimization manuals. <https://www.agner.org/optimize/>

CPU microarchitecture, instruction timing, and low-level performance notes [5].

Intel optimization manual. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

Intel x86 tuning and instruction behavior [9].

AMD Zen optimization guide. <https://developer.amd.com/resources/developer-guides-manuals/>

AMD-specific tuning for Zen-family processors [1].

31.1.6 *Survey and background papers. Floating-point arithmetic, Acta Numerica survey.* <https://www.cambridge.org/core/journals/acta-numerica/article/floating-point-arithmetic/287C4D5F6D4A43FBEEB1ABED2A405AAF>

Modern survey of floating-point arithmetic and IEEE behavior.

Gappa paper, Certification of bounds on expressions involving rounded operators. <https://arxiv.org/abs/cs/0701186>

Paper behind the Gappa tool.

Manuscript submitted to ACM

MPFR paper, A Multiple-Precision Binary Floating-Point Library with Correct Rounding. <https://inria.hal.science/inria-00070266v1/document>

Design of the usual high-precision oracle.

31.2 Paid books and standards

31.2.1 Best main book for elementary functions. *Elementary Functions: Algorithms and Implementation*, Jean-Michel Muller, Springer. <https://link.springer.com/book/10.1007/978-1-4899-7983-4>

Exp, log, trig functions, range reduction, polynomial approximation, table methods, and correct rounding.

31.2.2 Best main book for floating-point arithmetic. *Handbook of Floating-Point Arithmetic*, 2nd edition, Springer. <https://link.springer.com/book/10.1007/978-3-319-76526-6>

Floating-point formats, rounding, ULPs, FMA, exceptions, compiler issues, Gappa, MPFR, and formal verification.

31.2.3 Strong general book on finite precision error. *Accuracy and Stability of Numerical Algorithms*, Nicholas J. Higham, SIAM. <https://epubs.siam.org/doi/10.1137/1.9780898718027>

Numerical error, stability, and finite precision analysis, not libm design.

31.2.4 Shorter IEEE floating-point book. *Numerical Computing with IEEE Floating Point Arithmetic*, Michael L. Overton, SIAM. <https://epubs.siam.org/doi/book/10.1137/1.9780898718072>

Shorter IEEE floating-point introduction than the Handbook.

31.2.5 Historical libm implementation reference. *Software Manual for the Elementary Functions*, Cody and Waite. <https://www.amazon.com/Elementary-Functions-Prentice-Hall-computational-mathematics/dp/0138220646>

Classic libm structure: reduction, rational approximation, special cases, and implementation constants. Not a modern correctness target.

31.2.6 Official floating-point standard. *IEEE 754-2019 official standard page.* <https://standards.ieee.org/standard/754-2019.html>

Authoritative standard for floating-point formats, rounding attributes, exceptions, NaNs, infinities, signed zeros, subnormals, and required operations.

References

- [1] Advanced Micro Devices. [n. d.]. Software optimization guides for AMD processors. <https://developer.amd.com/resources/developer-guides-manuals/>. Accessed 2026-06-08.
- [2] Arm Limited. [n. d.]. Arm Optimized Routines. <https://www.arm.com/why-arm/technologies/compute-library>. Accessed 2026-06-08.
- [3] CORE-MATH Project. [n. d.]. CORE-MATH. <https://core-math.gitlabpages.inria.fr/>. Accessed 2026-06-08.
- [4] Catherine Daramy-Loirat, David Defour, Florent de Dinechin, Matthieu Gallet, Nicolas Gast, and Jean-Michel Muller. 2006. *CR-LIBM: A library of correctly rounded elementary functions in double-precision*. Technical Report. LIP. <https://ens-lyon.hal.science/ensl-01529804v1/document> Technical report describing version 0.10beta.
- [5] Agner Fog. [n. d.]. Optimization manuals. <https://www.agner.org/optimize/>. Accessed 2026-06-08.
- [6] Gappa. [n. d.]. Gappa user guide. <https://gappa.gitlabpages.inria.fr/gappa/index.html>. Accessed 2026-06-08.
- [7] David Goldberg. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *Comput. Surveys* 23, 1 (1991), 5–48. doi:10.1145/103162.103163
- [8] IEEE. 2019. IEEE Standard for Floating-Point Arithmetic. doi:10.1109/IEEESTD.2019.8766229
- [9] Intel Corporation. [n. d.]. Intel 64 and IA-32 Architectures Software Developer’s Manual. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed 2026-06-08.
- [10] Jean-Michel Muller. 2016. *Elementary Functions: Algorithms and Implementation* (3 ed.). Birkhäuser. doi:10.1007/978-1-4899-7983-4

- [11] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. 2018. *Handbook of Floating-Point Arithmetic* (2 ed.). Birkhäuser. doi:10.1007/978-3-319-76526-6
- [12] Oracle. [n. d.]. Numerical Computation Guide. https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html. Accessed 2026-06-08.
- [13] RLibm Project. [n. d.]. Rlibm project page. <https://people.cs.rutgers.edu/~sn349/rlibm/>. Accessed 2026-06-08.
- [14] SageMath Developers. [n. d.]. SageMath documentation. <https://doc.sagemath.org/>. Accessed 2026-06-08.
- [15] Naoki Shibata. 2018. SLEEF: A Portable Vectorized Library of Elementary Functions. In *2018 IEEE 25th International Conference on High Performance Computing Workshops (HiPCW)*. 49–54. doi:10.1109/HiPCW.2018.86345
- [16] Sollya. [n. d.]. Sollya. <https://www.sollya.org/>. Accessed 2026-06-08.
- [17] The MPFR Team. [n. d.]. GNU MPFR. <https://www.mpfr.org/>. Accessed 2026-06-08.
- [18] The Rocq Prover Development Team. [n. d.]. The Rocq Prover reference manual. <https://rocq-prover.org/refman>. Accessed 2026-06-08.

A Useful vocabulary

Term	Meaning
argument reduction	A mapping from the public input to a smaller interval where approximation is easier.
kernel	The exact function that is actually approximated after reduction.
transformed kernel	A kernel rewritten to remove a zero, cancellation point, or unstable relative error.
reduced interval	The interval on which the approximation bound is claimed.
residual	The small quantity that remains after a table lookup or other reduction step.
reconstruction	The step that combines the reduced result with exponents, table constants, signs, or quadrants.
minimax polynomial	A polynomial that minimizes the worst error on a stated interval.
coefficient format	The floating-point or fixed-width format in which the implementation stores the coefficients.
evaluation scheme	The operation order used to evaluate the polynomial, such as Horner or Estrin.
table invariant	The exact real relation that a table entry is meant to support.
exactness constraint	A requirement that one later operation be exact or tightly bounded because of how a constant was split.
faithful rounding	Returning one of the two floating-point neighbors of the exact result.
correct rounding	Returning the value selected by the active rounding mode from the exact real result.
hard case	An input whose exact result lies unusually close to a rounding boundary.
oracle	An independent high-precision reference used for validation, usually MPFR in this workflow.
fast path	The cheap path intended for ordinary inputs.
accurate path	The slower path used when the fast path cannot justify the final rounding decision.
ULP	The gap between consecutive floating-point values at a given magnitude.

B How the proof pieces fit together

Keep these obligations separate when reading or writing a proof note.

Manuscript submitted to ACM

- (1) **Reduction bound:** distance between the computed reduced argument and the exact reduced argument.
- (2) **Approximation bound:** distance between the mathematical polynomial and the kernel on the reduced interval.
- (3) **Evaluation bound:** rounding error from the compiled operation sequence that evaluates the polynomial.
- (4) **Reconstruction bound:** effect of split constants, exponent additions, sign fixes, and table lookups.
- (5) **Rounding argument:** margin to a rounding boundary, or the need for a fallback path.
- (6) **Oracle validation:** agreement with an independent reference over many inputs. Engineering evidence, not a proof certificate.

A small approximation error does not fix a bad reduction. A split table does not prove the polynomial. MPFR agreement does not certify the floating-point circuit. State which obligation each artifact covers.